Προγραμματισμός της Αναπτυξιακής πλακέτας Altera DE1-SoC ως Σύστημα Ελέγχου Κυκλοφορίας (Traffic Light Controller)

Παυλίνα Μοσχοπούλου
Τμήμα Μηχανικών Πληροφορικής
και Ηλεκτρονικών Συστημάτων
Διεθνές Πανεπιστήμιο Ελλάδος

***Περίληψη*—Η ρύθμιση της κυκλοφορίας των αυτοκινήτων αποτελεί βασικό κομμάτι του αστικού ιστού. Οι φωτεινοί σηματοδότες είναι αυτοί που κατά βάση χρησιμοποιούνται για τη διαχείριση της ροής της κυκλοφορίας και είναι κρίσιμοι για την ασφάλεια τόσο των οδηγών όσο και των πεζών. Η υλοποίησή τους μπορεί να γίνει με τη χρήση των FPGA. Στόχος της εργασίας είναι ο χειρισμός των φαναριών ενός φανταστικού κόμβου με τη χρήση των FPGA και η οπτικοποίηση του αποτελέσματος στα LEDs της αναπτυξιακής πλακέτας Altera DE1-SoC.**

Keywords—FPGA, Traffic Lights

I. ΕΙΣΑΓΩΓΗ

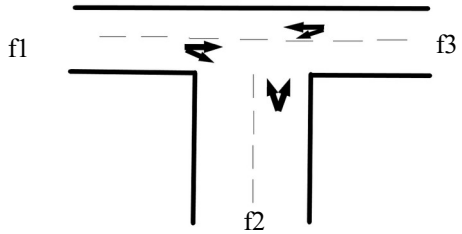
Οι φωτεινοί σηματοδότες ή αλλιώς τα γνωστά φανάρια, αποτελούν βασικό στοιχείο τόσο για την οδική ασφάλεια αλλά και την αποτελεσματική διαχείριση της κυκλοφορίας. Η υλοποίησή τους με τη χρήση των FPGA είναι ιδιαίτερα χρήσιμη καθώς επιτρέπει αξιόπιστη, παράλληλη και γρήγορη εκτέλεση σε πραγματικό χρόνο, καθώς και ευέλικτο σχεδιασμό.

Το λογισμικό που χρησιμοποιήθηκε για την υλοποίηση της εφαρμογής είναι το Quartus II 13.1, ένα ισχυρό λογισμικό για τον προγραμματισμό FPGA το οποίο μπορεί να υποστηρίξει είτε Verilog είτε VHDL. Διαθέτει γραφικό περιβάλλον σχεδίασης (Block Diagram Editor) όπου επιτρέπει τη δημιουργία κυκλωμάτων με τη χρήση σχηματικών διαγραμμάτων. Ακόμη, μετά την υλοποίηση του κώδικα μπορεί να αναπαραστήσει το εσωτερικό κύκλωμα που παράγεται στο FPGA ώστε να οπτικοποιηθούν οι συνδέσεις και ο χώρος που καταλαμβάνει η εκάστοτε εφαρμογή. Παρέχει μάλιστα εργαλεία προσομοίωσης για τον έλεγχο της σωστής λειτουργίας του κυκλώματος. Στην εργασία έχει χρησιμοποιηθεί η γλώσσα VHDL και η λειτουργικότητα του κώδικα πριν την μεταφορά του στην πλακέτα ελέγχθηκε μέσω του Waveform Simulation.

Η αναπτυξιακή πλακέτα που αξιοποιήθηκε για την επαλήθευση του κώδικα ήταν η Altera DE1-SoC. Διαθέτει το chip Cyclone V SoC το οποίο περιλαμβάνει έναν FPGA πυρήνα και ένα διπύρηνο επεξεργαστή ARM CORTEX-A9. Επιπλέον διαθέτει μια σειρά από μνήμες (DDR3, SDRAM Flash Memory), εισόδους/εξόδους (LEDs, switches, push buttons, GPIO headers) και υποστηρίζει διάφορα πρωτόκολλα επικοινωνίας (USB, UART, SD card). Αποτελεί μια κατάλληλη πλακέτα για εκπαιδευτικούς σκοπούς αλλά και για ενσωματωμένα συστήματα που απαιτούν έναν συνδυασμό υλικού και λογισμικού.

II. ΣΧΕΔΙΑΣΗ ΕΦΑΡΜΟΓΗΣ ΚΑΙ FINITE STATE MACHINE

Ο στόχος της εργασίας είναι η δημιουργία ενός συστήματος ελέγχου κυκλοφορίας, για αυτό πρώτο βήμα είναι ο καθορισμός του φανταστικού κόμβου και της ροής της κυκλοφορίας, αλλά και ο ρυθμός εναλλαγής μεταξύ των καταστάσεων των σηματοδοτών. Στην εικόνα 1 φαίνεται σχηματικά η μορφή του κόμβου.



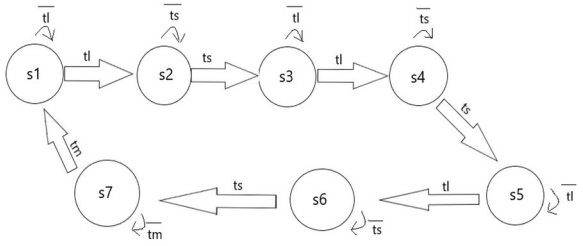
Εικ.1 Ο φανταστικός κόμβος

Ο πίνακας 1 αλλά και το Finite State Machine στην εικόνα 1 παρουσιάζουν την κάθε κατάσταση και τη διάρκειά της. Ως Μηχανή Πεπερασμένων Καταστάσεων (Finite State Machine ή FSM) ορίζεται ένα μοντέλο σχεδίασης συστημάτων που λειτουργούν μεταβαίνοντας από τη μία κατάσταση σε μία άλλη με βάση εισόδους ή χρονισμό.

Η κάθε κατάσταση συμβολίζεται ως s_i, με i=1...7 για τις 7 καταστάσεις που έχουν χρησιμοποιηθεί. Τα πρώτα τρία bits, συμβολίζουν το πορτοκαλί του κάθε φαναριού για τις ροές αυτοκινήτων f_1, f_2, f_3 αντίστοιχα. Τα τρία επόμενα, ισχύουν πάλι κατά σειρά για f_1, f_2, f_3 και συμβολίζουν ποια από τις τρεις ροές έχει πράσινο σηματοδότη, δηλαδή bit 1, με τις υπόλοιπες δύο να έχουν κόκκινο δηλαδή bit 0. Τέλος, τα τρία τελευταία bit συμβολίζουν τις τρεις διαβάσεις πεζών, οι οποίες πρασινίζουν ταυτόχρονα εφόσον έχει ολοκληρωθεί ένας κύκλος για τα φανάρια των αυτοκινήτων.

Οι χρόνοι διάρκειας έχουν επιλεχθεί σύμφωνα με τον προσωπικό Αριθμό Μητρώου 2020096, ως εξής:

- tl=9s, για τη διάρκεια όπου ένα φανάρι αυτοκινήτων είναι πράσινο ενώ τα υπόλοιπα παραμένουν κόκκινα
- ts=2s, η διάρκεια των πορτοκαλί φαναριών
- tm=6s, η διάρκεια όπου τα φανάρια των πεζών είναι πράσινα



Εικ 2. Finite State Machine

s1	000	001	000	9sec
s2	001	000	000	2sec
s3	000	010	000	9sec
s4	010	000	000	2sec
s5	000	100	000	9sec
s6	100	000	000	2sec
s7	000	000	111	6sec

Πίνακας 1

III. ΔΗΜΙΟΥΡΓΙΑ ΚΩΔΙΚΑ

Έχει δημιουργηθεί ένα πρότζεκτ με δύο αρχεία. Το πρώτο αρχείο, όπου ο κώδικάς του φαίνεται στην εικόνα 3, λειτουργεί ως διαιρέτης συχνότητας του εσωτερικού ρολογιού των 50Hz σε 1Hz, δηλαδή σε ένα tick το δευτερόλεπτο, για την ευκολότερη διαχείριση του χρονισμού. Το αρχείο ονομάζεται clockdivider, συνεπώς το ίδιο όνομα έχει και η οντότητα (entity) μέσα σε αυτό. Ορίζονται ως είσοδοι clk_in (το εσωτερικό ρολόι των 50Hz) και το reset, και ως έξοδος clk_out η νέα παραγόμενη συχνότητα του ενός Hz. Μέσα στη δομή του architecture, οργανώνεται η συμπεριφορά των σημάτων. Αρχικά ορίζονται τα δύο σήματα, counter και clk_sig και αρχικοποιούνται στο 0. Ρόλος του counter είναι να μετράει τα ticks του εσωτερικού ρολογιού και γι'αυτό ορίζεται ως integer. Το clk_sig θα λαμβάνει αποκλειστικά τις τιμές 0 και 1 καθώς θα είναι το σήμα της καινούριας συχνότητας, κι έτσι ορίζεται ως STD_LOGIC. Στη συνέχεια ορίζουμε μια διαδικασία (process), η οποία δέχεται ως ορίσματα το clk_in και το reset. Έτσι, η διαδικασία εκτελείται κάθε φορά που μεταβάλλεται η κατάσταση ενός εκ των δύο. Μέσω μίας if, εάν το reset ενεργοποιηθεί και πάρει την τιμή 1, τότε αρχικοποιείται στο 0 τόσο ο counter όσο και το σήμα του νέου ρολογιού clk_sig. Από την άλλη, όταν το clk_in αλλάζει κατάσταση και συγκεκριμένα βρίσκεται στο ανερχόμενο μέτωπο του παλμού, ελέγχεται η εξής συνθήκη: εάν ο counter έχει φτάσει τον αριθμό 24999999, το σήμα clk_sig αναστρέφεται και ο counter μηδενίζεται, διαφορετικά ο counter αυξάνει κατά ένα. Αυτό συμβαίνει, καθώς θέλουμε από τα 50MHz, δηλαδή 50.000.000 τικ το δευτερόλεπτο, να δημιουργήσουμε 1Hz δηλαδή ένα τικ το δευτερόλεπτο. Έτσι, στη μισή διάρκεια του αρχικού ρολογίου, δηλαδή στα 24999999 το clk_sig θα έχει τιμή είτε 0 είτε 1, και κατά την υπόλοιπη ημιπερίοδο θα έχει την αντίστροφη τιμή, δημιουργώντας έτσι μία πλήρη περίοδο του νέου σήματος κάθε 50 εκατομμύρια κύκλους του εσωτερικού ρολογιού, δηλαδή κάθε δευτερόλεπτο. Τέλος, το τελικό σήμα clk_sig που δημιουργείται το αναθέτουμε στην έξοδο clk_out την οποία θα χρησιμοποιήσουμε στο επόμενο αρχείο.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity clockdivider is
    Port (
        clk_in  : in  STD_LOGIC;   -- 50MHz
        reset   : in  STD_LOGIC;   -- Reset
        clk_out : out STD_LOGIC    -- 1Hz (1 tick/sec)

    );
end clockdivider;

architecture Behavioral of clockdivider is
    signal counter : integer := 0;
    signal clk_sig : STD_LOGIC := '0';
begin
    process(clk_in, reset)
    begin
        if reset = '1' then
            counter <= 0;
            clk_sig <= '0';
        elsif rising_edge(clk_in) then
            if counter = 24999999 then --debug ow 24999999 else 4
                clk_sig <= not clk_sig;
                counter <= 0;
            else
                counter <= counter + 1;
            end if;
        end if;
    end process;

    clk_out <= clk_sig;

end Behavioral;
```

Εικ.3 Ο κώδικας του clockdivider

Στη συνέχεια έχουμε το δεύτερο και βασικό αρχείο το οποίο διαχειρίζεται τη λειτουργία των φαναριών και έχει οριστεί ως top-level. Μέσα στην οντότητα ορίζονται ως έξοδοι το διάνυσμα lights των 9bit, που προσομοιώνουν το σύνολο των φαναριών, και το ρολόι που έχουμε δημιουργήσει για να μπορέσουμε να το ελέγξουμε στην προσομοίωση. Ως είσοδοι έχουν οριστεί το ρολόι του συστήματος και το reset. Στο architecture, ορίζουμε αρχικά ένα σήμα state τύπου state_type, το οποίο μπορεί να πάρει τις καταστάσεις από s1 έως s7 όπως τις ορίσαμε παραπάνω. Ορίζουμε, ακόμη, έναν μετρητή count 4-bit που θα μετράει τα δευτερόλεπτα, αλλά και τις σταθερές SEC2, SEC6 και SEC9 με τις τιμές των 2,6, και 9 δευτερολέπτων που θα χρειαστούμε.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TrafficLight is

   port (lights: out STD_LOGIC_VECTOR (8 downto 0);

         main_clk: in STD_LOGIC;

         reset : in STD_LOGIC;

         divided_clk_out : out STD_LOGIC --debug
         );

   end TrafficLight;


architecture Behavioral of TrafficLight is

   type state_type is (s1, s2, s3, s4, s5, s6, s7);
   signal state: state_type;
    signal count: unsigned(3 downto 0);
    constant SEC2: unsigned(3 downto 0) := "0010";
    constant SEC6: unsigned(3 downto 0) := "0110";
    constant SEC9: unsigned(3 downto 0) := "1001";

   signal divided_clk : std_logic;
```

Εικ.4 Δηλώσεις και αρχικοποιήσεις στο αρχείο TrafficLight

Έπειτα, μέσω του port map συνδέουμε τα τοπικά σήματα του αρχείου clockdivider σε νέα σήματα του αρχείου TrafficLight προκειμένου να αξιοποιηθούν για το χρονισμό και το reset.

```
begin


u1: entity work.clockdivider
      port map (
         clk_in => main_clk,
         clk_out => divided_clk,
         reset   => reset
      );
```

Εικ.5 Το port map μεταξύ των σημάτων

Μέσω μιας καινούριας διαδικασίας, χειριζόμαστε την εναλλαγή των καταστάσεων. Αρχικά σε περίπτωση reset, το σήμα count μηδενίζεται και η μεταβλητή state παίρνει την default τιμή s1. Διαφορετικά, εάν το ρολόι που δημιουργήσαμε βρίσκεται στο ανερχόμενο μέτωπο του παλμού, μέσω μίας case, εναλλάσσονται οι καταστάσεις του state: εάν το state είναι s1 και το count δεν έχει φτάσει στην τιμή του SEC9 που έχει οριστεί ως χρόνος, τότε παραμένει η ίδια κατάσταση και απλά το count αυξάνει κατά 1, ενώ έαν ο count βρίσκεται στην τιμή του SEC9 τότε το state αλλάζει στην επόμενη κατάσταση s2 και το count μηδενίζεται. Η ίδια διαδικασία ακολουθείται για όλες τις καταστάσεις με τα αντίστοιχα χρονικά διαστήματα (SEC2, SEC6, SEC9) όπως φαίνεται στις εικόνες 6 και 7, μέχρι και την κατάσταση s7, η οποία όταν περάσουν τα 6 δευτερόλεπτα που έχουν οριστεί επιστρέφει στην κατάσταση s1.

```
process (divided_clk, reset)
begin
   if reset = '1' then
      state <= s1;
      count <= X"0";

   elsif divided_clk'event and divided_clk = '1' then
   case state is
      when s1 =>
                  if count < SEC9 then
                     state <= s1;
                     count <= count + 1;
                  else
                     state <= s2;
                     count <= X"0";
                  end if;
      when s2 =>
                  if count < SEC2 then
                     state <= s2;
                     count <= count + 1;
                  else
                     state <= s3;
                     count <= X"0";
                  end if;

      when s3 =>
                  if count < SEC9 then
                     state <=s3;
                     count <= count + 1;
                  else
                     state <= s4;
                     count <= X"0";
                  end if;
```

Εικ.6 Η διαδικασία για την εναλλαγή κατάσεων με τη χρήση case

```
      when s4 =>
                  if count < SEC9 then
                     state <= s4;
                     count <= count + 1;
                  else
                     state <= s5;
                     count <= X"0";
                  end if;

      when s5 =>
                  if count < SEC2 then
                     state <= s5;
                     count <= count + 1;
                  else
                     state <=s6;
                     count <= X"0";
                  end if;

      when s6 =>
                  if count <= SEC9 then
                     state <= s6;
                     count <= count + 1;
                  else
                     state <=s0;
                     count <= X"0";
                  end if;

      when others =>
                     state <= s0;
   end case;
  end if;
end process;
```

Εικ7. Ο υπόλοιπος κώδικας της διαδικασίας εναλλαγής καταστάσεων

Τέλος, μέσω μίας ακόμα διαδικασίας, της c2, ορίζουμε την τιμή που παίρνει το σήμα lights για κάθε state όπως φαίνεται και στον πίνακα 1. Πιο συγκεκριμένα, η διαδικασία αυτή εκτελείται κάθε φορά που αλλάζει η τιμή της μεταβλητής state μέσα από την προηγούμενη case, και αντιστοιχίζει κάθε κατάσταση σε μία προκαθορισμένη τιμή του σήματος lights. Η τελική εντολή when others εξασφαλίζει ότι σε περίπτωση που το state πάρει μια απρόσμενη τιμή, το σύστημα θα δώσει ως έξοδο την default τιμή του s1.

```
C2: process (state)
    begin
    case state is
         when s1 => lights <= "000001000";
         when s2 => lights <= "001000000";
         when s3 => lights <= "000010000";
         when s4 => lights <= "010000000";
         when s5 => lights <= "000100000";
         when s6 => lights <= "100000000";
         when s7 => lights <= "000000111";
         when others => lights <= "000001000";
end case;
end process;

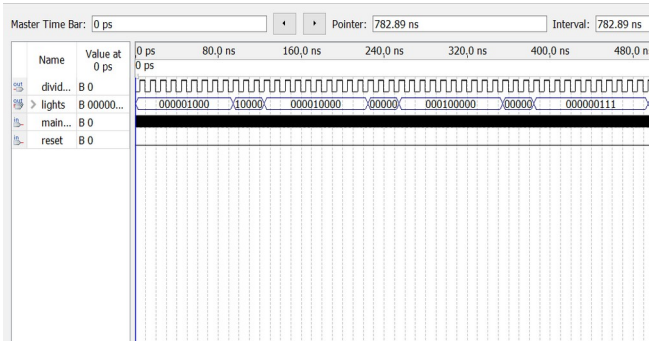
divided_clk_out <= divided_clk; -- debug

end Behavioral;
```

Εικ.8 Ανάθεση τιμών στα LEDs ανάλογα με την κατάσταση

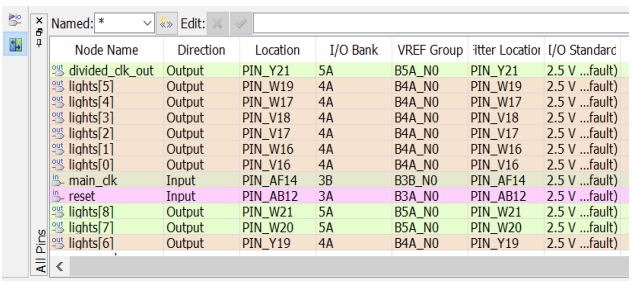
IV. ΠΡΟΣΟΜΟΙΩΣΗ ΚΑΙ ΑΠΟΤΕΛΕΣΜΑΤΑ

Ο κώδικας ελέγχθηκε μέσω του Waveform Simulation αφότου ρυθμίστηκε κατάλληλα το ρολόι για τους σκοπούς της προσομοίωσης και τα αποτελέσματα της εναλλαγής των καταστάσεων φαίνονται στην εικόνα 9. Διακρίνεται το διαιρεμένο ρολόι divided_clk, και μπορεί να παρατηρηθεί πως κάθε 9,6 και 2 παλμούς αντίστοιχα για την κάθε περίπτωση γίνεται η κατάλληλη εναλλαγή.



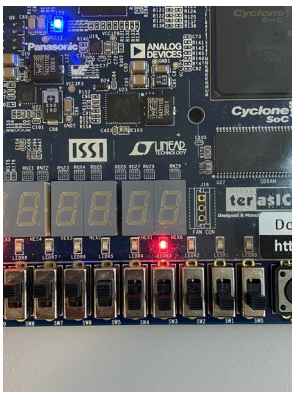
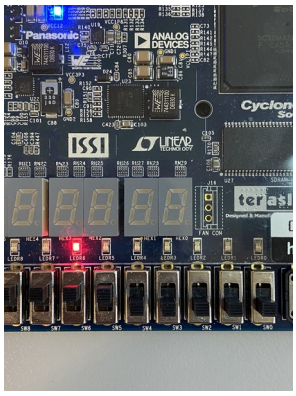
Εικ.9 Προσομοίωση στο Waveform Simulation

Τέλος, με τη βοήθεια του manual της πλακέτας συμπληρώθηκαν στο pin planner όλα τα pin εισόδων και εξόδων της πλακέτας όπως φαίνεται στην εικόνα 10. Ως reset ορίστηκε ένα slide switch, και οι έξοδοι των lights είναι τα led0 έως led8.

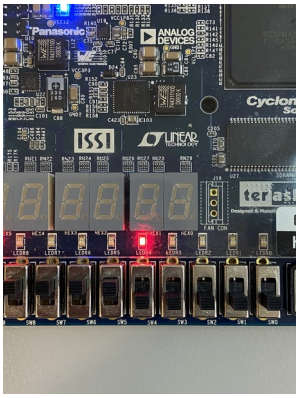
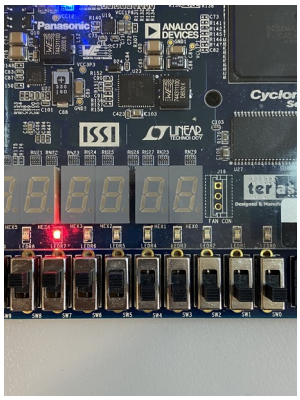


Εικ.10 Συμπλήρωση του pin planner

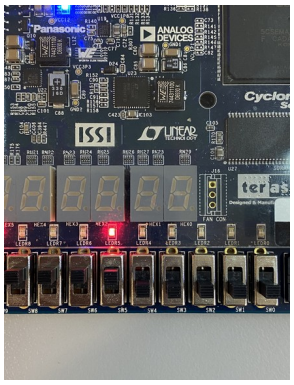
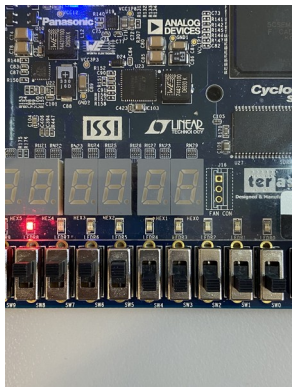
Στις εικόνες 11 έως 17 φαίνεται το τελικό αποτέλεσμα των επτά καταστάσεων στα leds της πλακέτας.

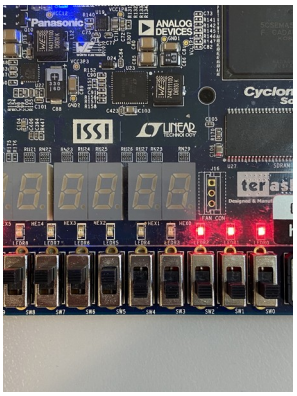
Εικ 11-12. Οι καταστάσεις s1 και s2

Εικ 13-14. Οι καταστάσεις s3 και s4

Εικ 15-16. Οι καταστάσεις s5 και s6



Εικ 7. Η κατάσταση s7